

Returning a Result from a Function



Algoritma dan Pemrograman II

Team Teaching:
Lia Farhatuaini, M.Kom
Sokid, M.Kom

INSTRUKSI RETURN



- pada pertemuan sebelumnya, kita belajar bagaimana fungsi menghasilkan suatu text dan menampilkannya pada console.
- fungsi juga bisa menghasilkan suatu hasil (result)
- untuk membuat suatu fungsi dapat mengembalikan hasil, kita membutuhkan instruksi **return**
- kata **return** merupakan suatu keyword python
- instruksi **return** memiliki 2 jenis yang berbeda:
 - return tanpa ekspresi
 - return dengan ekspresi

RETURN TANPA EKSPRESI



Perhatikan dan jalankan kode di bawah ini:

```
1  def selamat_ulang_tahun(harapan=True):  
2      print("Tiga...")  
3      print("Dua...")  
4      print("Satu...")  
5      if not harapan:  
6          return  
7  
8      print("Selamat Ulang Tahun")  
9  
10  selamat_ulang_tahun() #Pemanggilan tanpa argumen
```

```
10  selamat_ulang_tahun(False)
```

Kode disamping memiliki instruksi return yang tidak memiliki ekspresi setelahnya. return tersebut berada di dalam if. jika kondisi if terpenuhi, dalam hal ini harapan=False maka fungsi akan berhenti (*terminate*) tanpa mengembalikan nilai apapun dan tidak menjalankan perintah print yang berada di bawahnya.

RETURN DENGAN EKSPRESI-1



- Ada 2 keadaan ketika kita menggunakan return dengan ekspresi:
 - Pemberhentian eksekusi fungsi (seperti pada contoh sebelumnya)
 - fungsi akan mengevaluasi nilai ekspresi dan mengembalikannya (*return*) sebagai hasil dari fungsi

```
1  def fungsi_malas():  
2      return 123  
3  
4  x = fungsi_malas()  
5  print("Hasil dari fungsi malas adalah ", x)
```

- pada line 4 fungsi_malas dipanggil, maka line 1-2 akan dieksekusi, pada line 2, fungsi mengembalikan nilai 123 sebagai hasil dari fungsi. hasil tersebut dikembalikan ke line 4 dan disimpan ke variabel x. variabel x kemudian dipanggil pada print untuk ditampilkan di console.

RETURN DENGAN EKSPRESI-2



- Kita bisa juga mengabaikan kembalian/hasil dari fungsi:

```
1 def fungsi_malas():  
2     print("aku lagi mode malas")  
3     return 123  
4  
5 print("Mata kuliah ini seru banget!")  
6 fungsi_malas()  
7 print("Mata kuliah ini boring banget...")
```

- ketika fungsi malas dipanggil di line 6, maka fungsi_malas line 1-3 akan di eksekusi. fungsi tersebut akan menjalankan print dan mengembalikan nilai 123, tetapi nilai tersebut tidak disimpan ke dalam variabel karena kita tidak membutuhkan nilai tersebut pada program kita untuk digunakan kembali.

NILAI NONE



- None merupakan suatu keyword pada python.
- Ada 2 keadaan yang memungkinkan None dapat digunakan dengan aman:
 - ketika dimasukkan (*assign*) ke dalam variabel, atau dikembalikan sebagai hasil fungsi.
 - ketika dibandingkan dengan suatu variabel untuk mengetahui keadaan internalnya.
- ketika suatu fungsi tidak mengembalikan suatu nilai menggunakan instruksi `return`, maka diasumsikan secara implisit fungsi tersebut me-return None.

```
1  def fungsi_aneh(n):  
2      if(n % 2 == 0):  
3          return True  
4  
5  
6  print(fungsi_aneh(12))  
7  print(fungsi_aneh(13))
```

LIST DAN FUNGSI



- Suatu list dapat di pass ke pada fungsi sebagai argumen:
- Suatu list dapat menjadi hasil dari fungsi:

```
1  def penjumlahan_list(lst):  
2      s = 0  
3  
4      for elemen in lst:  
5          s += elemen  
6  
7      return s  
8  
9  print(penjumlahan_list([10, 13, 15]))
```

- coba ganti kode menjadi:

```
9  print(penjumlahan_list(7))
```

```
1  def fungsi_list_aneh(n):  
2      list_aneh = []  
3  
4      for i in range(0, n):  
5          list_aneh.insert(0,i)  
6  
7      return list_aneh  
8  
9  print(fungsi_list_aneh(5))
```

KUIS 23



- buatlah sebuah kode python yang di dalamnya terdapat fungsi untuk mengecek tahun kabisat. fungsi tersebut memiliki satu argumen (tahun) dan akan mengembalikan nilai True jika tahun merupakan tahun kabisat dan mengembalikan nilai False jika sebaliknya.
- kode terdiri dari 2 list data: list data_uji yang berisi data tahun yang akan diisi dan list data_hasil berisi hasil yang diharapkan.
- kode akan memberi tahu jika hasil dari program kamu berhasil atau tidak.

```
1  def tahun_kabisat(tahun):
2      #lanjutkan kode
3      return #lanjutkan kode
4      #lanjutkan kode
5
6  data_uji = [1900, 2000, 2016, 1987]
7  data_hasil = [False, True, True, False]
8
9  for i in range(len(data_uji)):
10     th = data_uji[i]
11     print(th,"->", end="")
12     hasil = tahun_kabisat(th)
13     if hasil == data_hasil[i]:
14         print("Ok")
15     else:
16         print("Gagal")
```

KUIS 24



- lanjutan dari program kuis 23, buatlah sebuah fungsi yang memiliki 2 argumen (tahun, bulan) dan mengembalikan jumlah hari dari pasangan tahun-bulan yang diberikan.

```
1  def tahun_kabisat(tahun):
2      #lanjutkan kode
3      return #lanjutkan kode
4      #lanjutkan kode
5
6  def hari_didalam_bulan(tahun, bulan):
7      #lanjutkan kode
8      return #lanjutkan kode
9      #lanjutkan kode
10
11 data_uji = [1900, 2000, 2016, 1987]
12 data_bulan = [2, 2, 1, 11]
13 data_hasil = [28, 29, 31, 30]
14
15 for i in range(len(data_uji)):
16     thn = data_uji[i]
17     bln = data_bulan[i]
18     print(thn, bln, "->", end="")
19     hasil = hari_didalam_bulan(thn, bln)
20     if hasil == data_hasil[i]:
21         print("Ok")
22     else:
23         print("Gagal")
```

KUIS 25



- lanjutan dari program kuis 24, buatlah sebuah fungsi yang memiliki 3 argumen (tahun, bulan, hari) dan mengembalikan hari dari tahun yang sesuai atau mengembalikan None jika tidak ada argumen yang valid.
- gunakan fungsi test sebelumnya dan sesuaikan dengan masalah

```
1  def tahun_kabisat(tahun):
2      #lanjutkan kode
3      return #lanjutkan kode
4      #lanjutkan kode
5
6  def hari_didalam_bulan(tahun, bulan):
7      #lanjutkan kode
8      return #lanjutkan kode
9      #lanjutkan kode
10
11 def hari_pada_tahun(tahun, bulan, hari):
12     #lanjutkan kode
13     return #lanjutkan kode
14     #lanjutkan kode
15 print(hari_pada_tahun(2000, 12, 31))
```

KUIS 26



- bilangan asli dikatakan prima jika lebih besar dari 1 dan tidak memiliki pembagi selain 1 dan dirinya sendiri.
- buatlah kode python, yang di dalamnya terdapat fungsi untuk mengecek bilangan prima.
 - nama fungsinya: cek_prima
 - memiliki satu argumen (nilai untuk dicek)
 - mengembalikan True jika argumen merupakan bilangan prima, dan False sebaliknya.

```
1  def cek_prima(bilangan):
2      #Tulis kode di sini
3
4      for i in range(1,20):
5          if cek_prima(i + 1):
6              print(i + 1, end=" ")
7      print()
```

KUIS 27



- bilangan asli dikatakan prima jika lebih besar dari 1 dan tidak memiliki pembagi selain 1 dan dirinya sendiri.
- buatlah kode python, yang di dalamnya terdapat fungsi untuk mengecek bilangan prima.
 - nama fungsinya: cek_prima
 - memiliki satu argumen (nilai untuk dicek)
 - mengembalikan True jika argumen merupakan bilangan prima, dan False sebaliknya.

```
1  def cek_prima(bilangan):
2      #Tulis kode di sini
3
4  for i in range(1,20):
5      if cek_prima(i + 1):
6          print(i + 1, end=" ")
7  print()
```

- konsumsi BBM di Eropa dinyatakan dalam liter bahan bakar per 100 km perjalanan. di Amerika, konsumsi BBM dinyatakan dalam satuan mil per galon (mpg). tugas anda adalah membuat program untuk mengkonversi 1/100 km ke dalam satuan mpg dan sebaliknya.
- ketentuan:
 - terdapat dua buah fungsi dengan nama: Liter100km_ke_mpg dan mpg_ke_Liter100km
 - memiliki satu argumen dengan nama disesuaikan dengan nilainya.
 - 1 mil = 1609.344 meter
 - 1 galon = 3.785411784 liter

KUIS 28



```
1 def Liter100km_ke_mpg(liter):
2     #tulis kode disini
3     #
4     return mil / galon
5
6 def mpg_ke_Liter100km(mil):
7     #
8     #
9     return liter / km100
10
11 print(Liter100km_ke_mpg(3.9))
12 print(Liter100km_ke_mpg(7.5))
13 print(Liter100km_ke_mpg(10.))
14 print(mpg_ke_Liter100km(60.3))
15 print(mpg_ke_Liter100km(31.4))
16 print(mpg_ke_Liter100km(23.5))
```

- lengkapi kode disamping

- output yang diharapkan:

```
60.31143162393162
31.361944444444444
23.521458333333333
3.9007393587617467
7.490910297239916
10.009131205673757
```